Rockchip Application Notes VendorStorage

ID: RK-KF-YF-C35

Release Version: V1.4.0

Release Date: 2026-06-06

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This article describes the Vendor storage interface and ID definition.

Product Version

Chipset	Kernel Version
RockChip	kernel-4.19 kernel-5.10 kernel-6.1

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	赵仪峰/吴良清	2025-01-20	Initial version
V1.1.0	赵仪峰/吴良清	2025-01-24	Add customer reservation ID description
V1.2.0	周为新	2025-05-09	Add ID define for function multi-module field
V1.3.0	赵仪峰	2025-07-03	Add SPI NAND layout and fix SPI NOR layout
V1.4.0	Wang Hang	2026-06-06	Add ID define for search function field

Contents

Rockchip Application Notes VendorStorage

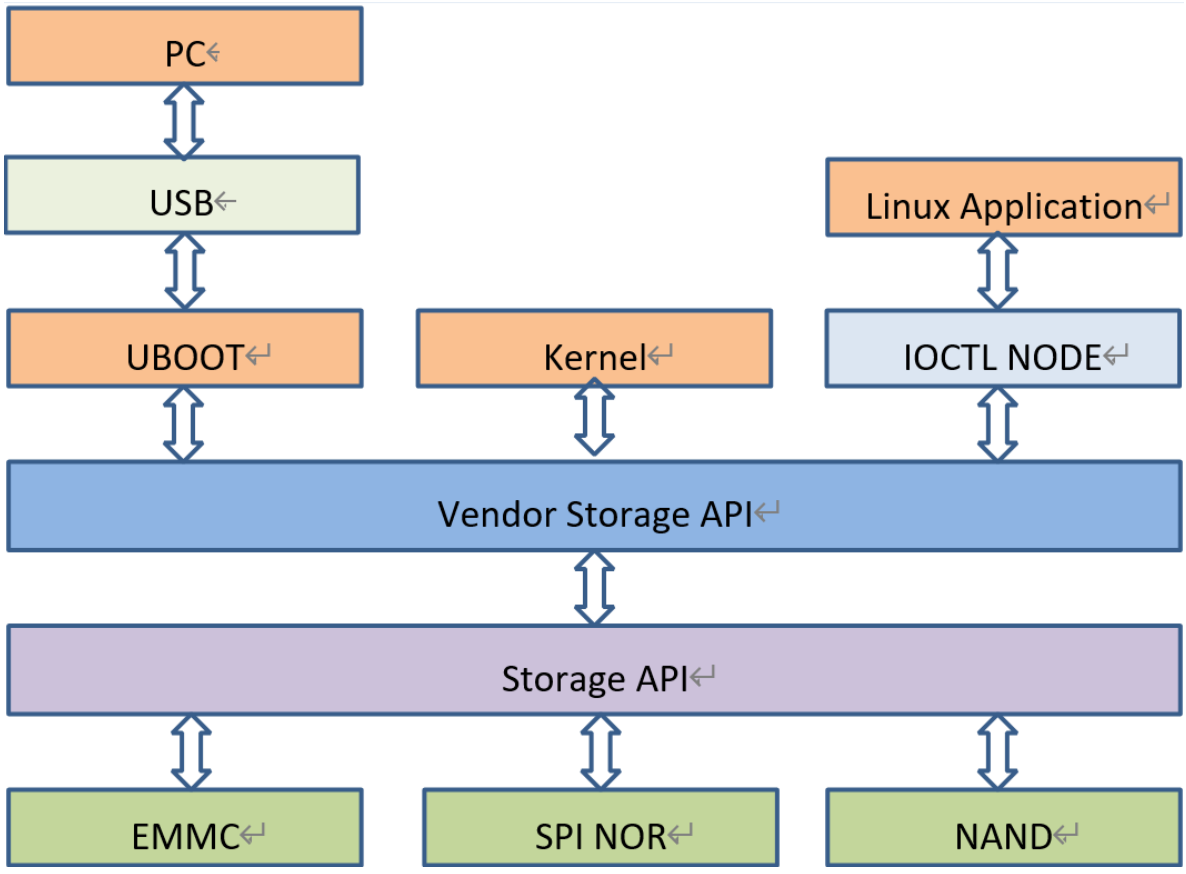
1. Application Note Summary
2. Vendor Storage Architecture
3. Data Layout
   - 3.1 SPI NOR Data Layout
   - 3.2 Raw NAND and SPI NAND Data Layout
   - 3.3 EMMC Data Layout
   - 3.4 Data Structure
   - 3.5 ID Define
4. API For UBOOT
   - 4.1 API For NON GKI kernel
   - 4.2 API For GKI kernel
   - 4.3 API for Android
5. PC Demo Tool
6. PC Tool Source Code
7. Linux Application Demo Code

1. Application Note Summary

Vendor storage is designed for stored SN, MAC and other vendor data.
Feature:

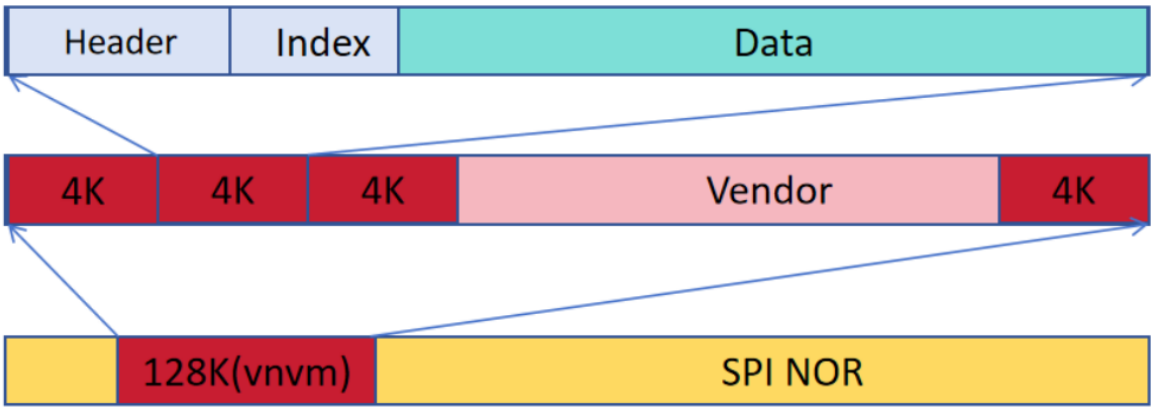
- Unique ID Access
- Reliable Data Validation
- Power Lost Recovery
- Writable and Readable for PC
- Writable and Readable for UBOOT
- Writable and Readable for Kernel
- Writable and Readable for Linux Application

2. Vendor Storage Architecture

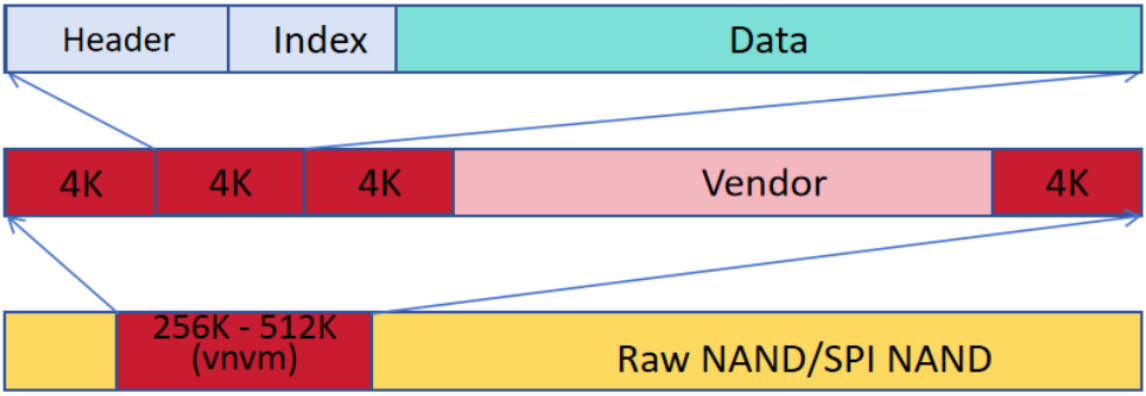


3. Data Layout

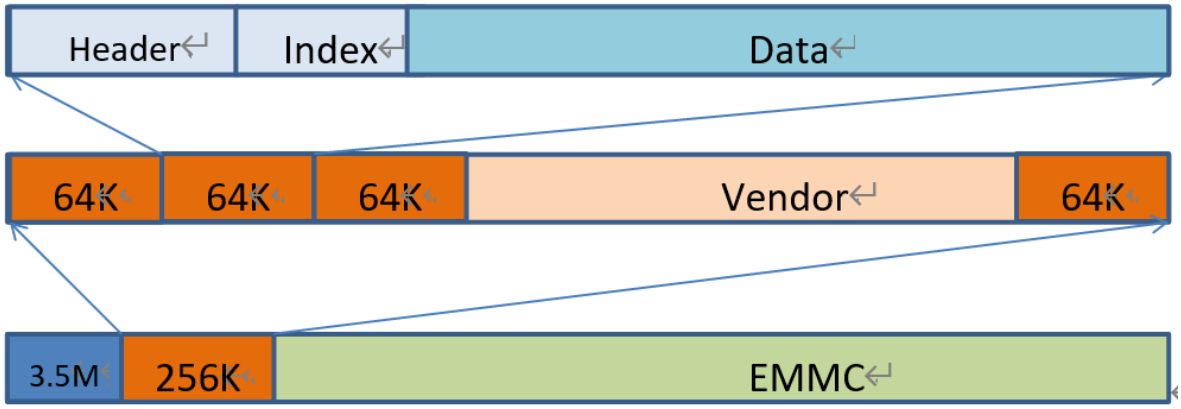
3.1 SPI NOR Data Layout



3.2 Raw NAND and SPI NAND Data Layout



3.3 EMMC Data Layout



3.4 Data Structure

Offset(B)	Size(B)	Name
0x0000	0x0004	Tag, the value is 0x524B5644.
0x0004	0x0004	Version, increase after write
0x0008	0x0002	Next index
0x000A	0x0002	Total items
0x000C	0x0002	Free offset
0x000E	0x0002	Free size
0x0010	0x0002	Item0->id
0x0012	0x0002	Item0->offset
0x0014	0x0002	Item0->size
0x0016	0x0002	Item0->flag(reversed)
0x0018	0x0008	Item1
…	…	Item x
---	---	Data
Size – 0x08	0x0004	Hash
Size – 0x04	0x0004	Version2, the value is the same as the Version

3.5 ID Define

ID	Function	Description
1	SN	Serial Number
2	WIFI MAC	WIFI MAC Address
3	LAN MAC	Ethernet MAC Address
4	BT MAC	Bluetooth MAC Address
5	HDMITX HDCP1.4 KEY	HDMITX HDCP1.4 KEY
6	DP HDCP1.4 KEY	DP HDCP1.4 KEY
7	HDCP_2X_ID	
8	DRM_KEY_ID	
9	PLAYREADY_CERT_ID	
10	ATTENTION_KEY_ID	
11	PLAYREADY_ROOT_KEY_0_ID	
12	PLAYREADY_ROOT_KEY_1_ID	
13	HDMIRX HDCP1.4 KEY	HDMIRX HDCP1.4 KEY
14	SENSOR_CALIBRATION_ID	
15	IMEI_ID	
16	LAN_RGMII_DL_ID	
17	EINK_VCOM_ID	E-ink Screen VCOM Voltage

ID	Function	Description
18		
19		
20	HDMIRX HDCP2.3 KEY	HDMIRX HDCP2.3 KEY
...		
60	sr	Stores the Super-Resolution online authorization key, corresponding to the offline key position +1000, i.e., 1060
61	Super-Resolution Algorithm Reserved Field / HDCP2.x RKIV	Super-Resolution algorithm reserved field, corresponding to offline reserved 1061/RKIV is a 16-byte encryption key for HDMI2.x firmware, providing additional security protection. Currently, some customers of 3528 and 356x use this additional encryption scheme.
62	memc	Stores the MEMC online authorization key, corresponding to the offline key position +1000, i.e., 1062
63	MEMC Algorithm Reserved Field	MEMC algorithm reserved field, corresponding to offline reserved 1063
64	pq	Stores the PQ online authorization key, corresponding to the offline key position +1000, i.e., 1064
65	PQ Algorithm Reserved Field	PQ algorithm reserved field, corresponding to offline reserved 1065
66	RKP Reserved Field	Used by RKP CSR, serial number when generating CSR
67	RKP Reserved Field	Used by RKP CSR, content of generated CSR
68	multi-module	Stores the multi-module online authorization key, corresponding to the offline key position +1000, i.e., 1068
69	multi-module Reserved Field	multi-module reserved field, corresponding to offline reserved 1069
70	touch_read	Stores the touch reading online authorization key, corresponding to the offline key position +1000, i.e., 1070
71	touch_read Algorithm Reserved Field	Touch reading algorithm reserved field, corresponding to offline reserved 1071
72	rknpu_ocr	Stores the OCR algorithm online authorization key, corresponding to the offline key position +1000, i.e., 1072

ID	Function	Description
73	rknpu_ocr Algorithm Reserved Field	OCR algorithm reserved field, corresponding to offline reserved 1073
74	face	Stores the face unlock online authorization key, corresponding to the offline key position +1000, i.e., 1074
75	Face Unlock Algorithm Reserved Field	Face unlock algorithm reserved field, corresponding to offline reserved 1075
76	knowledge	Offline key reserved field
77	knowledge Algorithm Reserved Field	Knowledge algorithm reserved field, corresponding to offline reserved 1077
...		
84	asr	Stores the ASR voice-to-text algorithm online authorization key, corresponding to the offline key position +1000, i.e., 1084
85	asr Algorithm Reserved Field	ASR algorithm reserved field, corresponding to offline reserved 1085
86	llm	Stores the LLM translation online authorization key, corresponding to the offline key position +1000, i.e., 1086
87	llm Algorithm Reserved Field	LLM translation algorithm reserved field, corresponding to offline reserved 1087
88	tts	Stores the TTS text-to-speech algorithm online authorization key, corresponding to the offline key position +1000, i.e., 1088
89	tts Algorithm Reserved Field	TTS algorithm reserved field, corresponding to offline reserved 1089
...		
96	aieraser	Stores the AI Eraser algorithm online authorization key, corresponding to the offline key position +1000, i.e., 1096
97	aieraser Algorithm Reserved Field	AI Eraser algorithm reserved field, corresponding to offline reserved 1097
98	search	Stores the AI Search algorithm online authorization key, corresponding to the offline key position +1000, i.e., 1098
99	search Algorithm Reserved Field	AI Search algorithm reserved field, corresponding to offline reserved 1099
...		

ID	Function	Description
156	customer reservation	Reserved for customer use, not for internal use by RK
...	customer reservation	Reserved for customer use, not for internal use by RK
256	customer reservation	Reserved for customer use, not for internal use by RK
1060	Super-Resolution Offline Key Position	Super-Resolution offline key position, -1000 equals online key position, i.e., 60
1061	Super-Resolution Offline Key Reserved Field	Super-Resolution offline key reserved field
1062	MEMC Offline Key Position	MEMC offline key position, -1000 equals online key position, i.e., 62
1063	MEMC Offline Key Reserved Field	MEMC offline key reserved field
1064	PQ Offline Key Position	PQ offline key position, -1000 equals online key position, i.e., 64
1065	PQ Offline Key Reserved Field	PQ offline key reserved field
1068	multi-module Offline Key Position	multi-module offline key position, -1000 equals online key position, i.e., 68
1069	multi-module Offline Key Reserved Field	multi-module offline key reserved field
1070	Touch Read Offline Key Position	Touch Read offline key position, -1000 equals online key position, i.e., 70
1071	Touch Read Offline Key Reserved Field	Touch Read offline key reserved field
1072	OCR Offline Key Position	OCR offline key position, -1000 equals online key position, i.e., 72
1073	OCR Offline Key Reserved Field	OCR offline key reserved field
1074	Face Offline Key Position	Face offline key position, -1000 equals online key position, i.e., 74
1075	Face Offline Key Reserved Field	Face offline key reserved field
1076	Knowledge Offline Key Position	Knowledge offline key position, -1000 equals online key position, i.e., 76
1077	Knowledge Offline Key Reserved Field	Knowledge offline key reserved field
1084	ASR Offline Key Position	ASR offline key position, -1000 equals online key position, i.e., 84
1085	ASR Offline Key Reserved Field	ASR offline key reserved field

ID	Function	Description
1086	LLM Offline Key Position	LLM offline key position, -1000 equals online key position, i.e., 86
1087	LLM Offline Key Reserved Field	LLM offline key reserved field
1088	TTS Offline Key Position	TTS offline key position, -1000 equals online key position, i.e., 88
1089	TTS Offline Key Reserved Field	TTS offline key reserved field
1096	AI Eraser Algorithm Offline Key Position	AI Eraser Algorithm offline key position, -1000 equals online key position, i.e., 96
1097	AI Eraser Algorithm Offline Key Reserved Field	AI Eraser Algorithm offline key reserved field
1098	AI Search Algorithm Offline Key Position	AI Search Algorithm offline key position, -1000 equals online key position, i.e., 98
1099	AI Search Algorithm Offline Key Reserved Field	AI Search Algorithm offline key reserved field

Note:
Data is in accordance with the 64 - bytes alignment stored in the NVM, this suggests that a minimum allocation for each item is 64 bytes, so write 1-64 bytes, the space will be allocated as 64 bytes, write 65-128 bytes, the space will be allocated as 128 bytes.

4. API For UBOOT

- Source code

u-boot/board/rockchip/common/storage/storage.c

```
int vendor_storage_init(void)
function: Initialize vendor storage
input : none
return: 0: Initialize success
        other: Initialize fail

int vendor_storage_read(u32 id, void *pbuf, u32 size)
function: read vendor storage by id
input : id, item id;
pbuf, data buffer;
size, number byte to read.
return: other: number byte have read.
        -1: read fail.

int vendor_storage_write(u32 id, void *pbuf, u32 size)
function: write vendor storage by id
input : id: item id;
pbuf: data buffer;
size: number bytes to write.
```

```
return:  0: write success
         other: write fail
```

4.1 API For NON GKI kernel

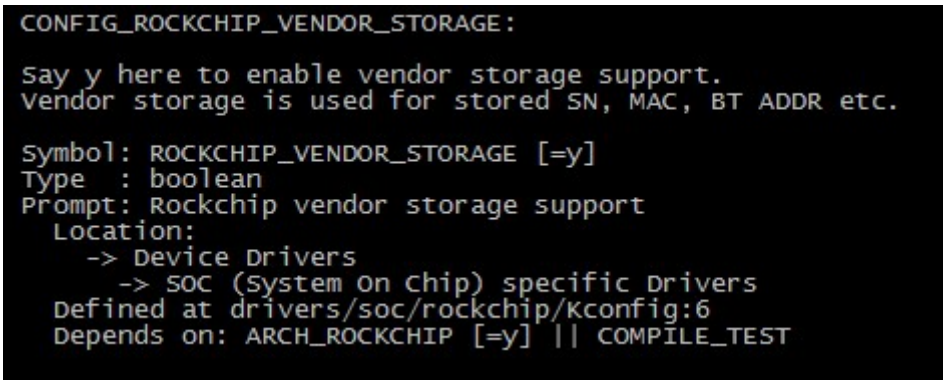
- Source code

kernel/drivers/soc/rockchip/rk_vendor_storage.c

```
int rk_vendor_read(u32 id, void *pbuf, u32 size)
function: read vendor storage by id
input : id: item id;
pbuf: data buffer;
size: number bytes to read.
return: other: number byte have read.
        -1: read fail

int rk_vendor_write(u32 id, void *pbuf, u32 size)
function: write vendor storage by id
input : id, item id;
pbuf: data buffer;
size: number bytes to write.
return: 0: write success
        other : write fail
```

- Menuconfig



4.2 API For GKI kernel

Due to the kernel interface limitations of GKI, the kernel needs to be added to ram_vendor_storage.ko. Only then can Vendor storage support the read interface, but not the write interface.

4.3 API for Android

- Source code

hardware/rockchip/libvendor_storage/vendor_storage.c

```
/*
 * vendor_storage_init must be call first
```

```
 * return:  ret=0 succes   ret<0 failed
 */
int vendor_storage_init(void);

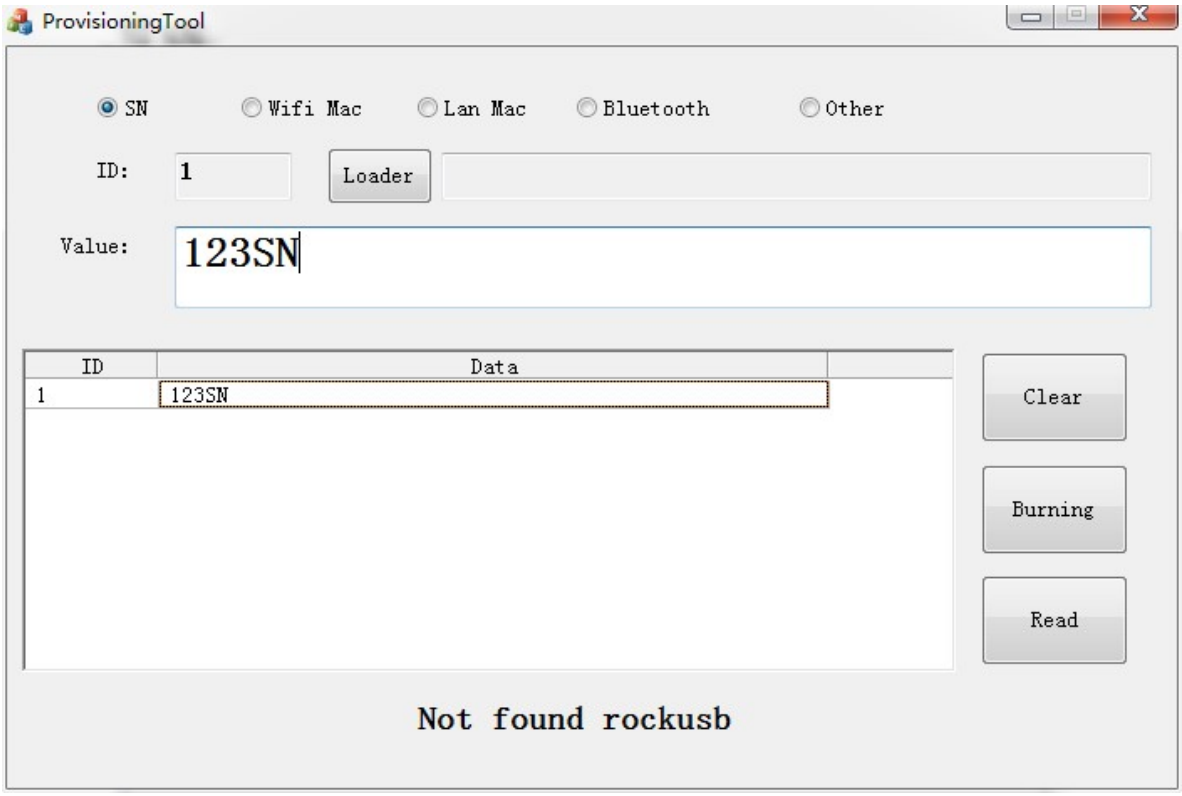
/*
 * emmc_vendor_write
 * id: item id
 * pbuf: write data
 * size: vendor_item.size
 * return:  ret>0 succes   ret<0 failed
 */
int emmc_vendor_write(uint32 id, void *pbuf, uint32 size);

/*
 * emmc_vendor_read
 * id: item id
 * pbuf: read data
 * size: vendor_item.size
 * return:  ret>0 succes   ret<0 failed
 */
int emmc_vendor_read(uint32 id, void *pbuf, uint32 size);
```

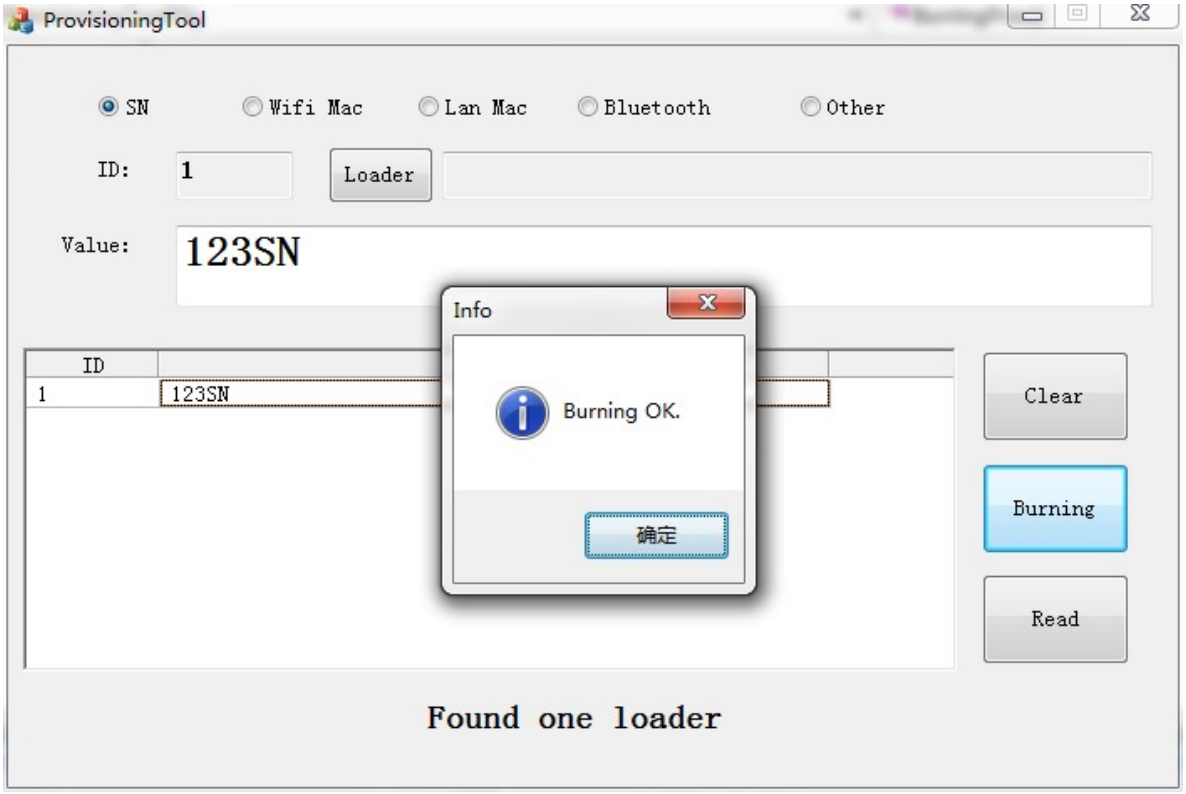
For specific usage, you can refer to hardware/rockchip/libvendor_storage/vendor_storage_test.c.

5. PC Demo Tool

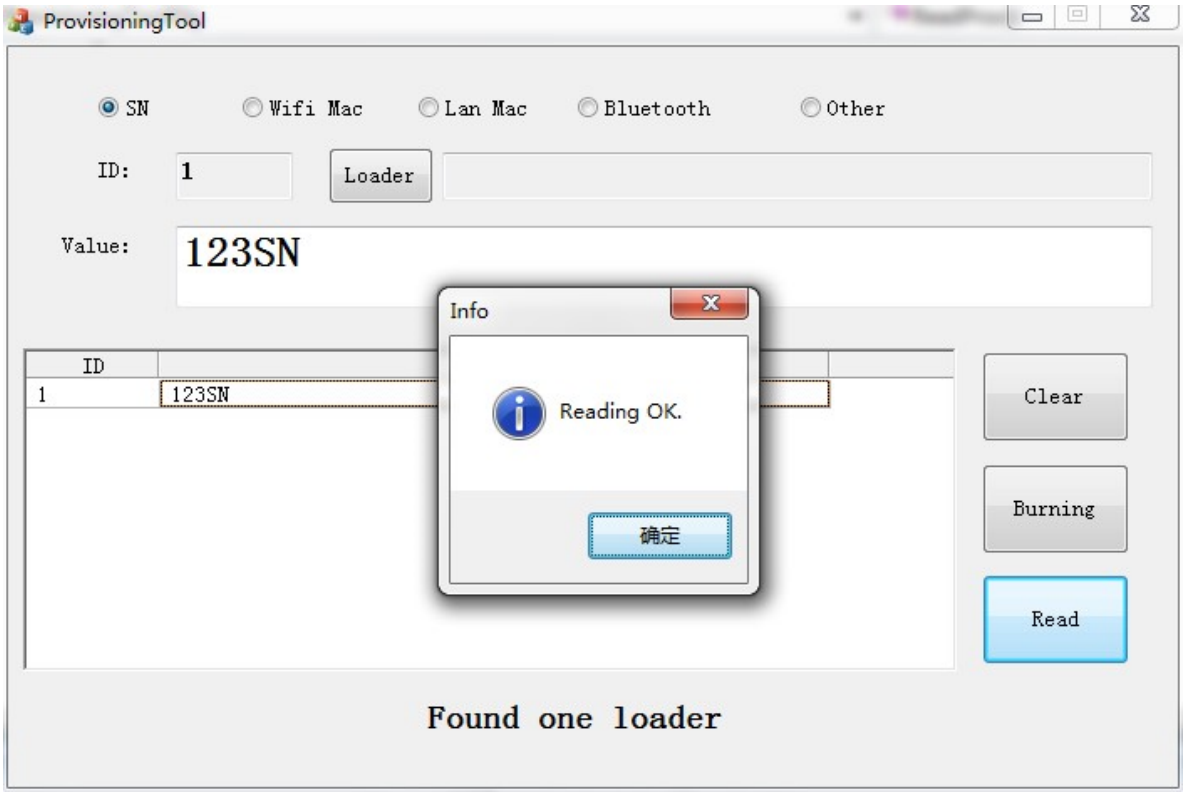
- Select SN option and input SN value, press ENTER key to add SN item to write request list



- Press Burning button will write data to vendor storage



- Select SN option and press Read button to read data form vendor storage



6. PC Tool Source Code

- PC Tool Read Data

```
/*--------------------------------------------------------------------
Name    : RK_ReadProvisioningData
Desc    : Read provisioning data by id
Params  : (IN)nID: id
          (OUT)pDataBuffer: buffer to save data, malloc by caller
          (IN|OUT)nBufferSize: in = size of buffer,out = actual data size
          (IN)dwLayer:device layer
Return  : TRUE: SUCCESSED
          FALSE: FAILED
--------------------------------------------------------------------*/
BOOL RK_ReadProvisioningData(USHORT nID,PBYTE pDataBuffer,USHORT
&nBufferSize,DWORD dwLayer=0);
```

- PC Tool Write Data

```
 /*--------------------------------------------------------------------
Name    : RK_WriteProvisioningData
Desc    : Write provisioning data by id
Params  : (IN)nID: id
          (IN)pDataBuffer: buffer to save data, malloc by caller
          (IN)nBufferSize: size of buffer
          (IN)dwLayer:device layer
Return  : TRUE: SUCCESSED
          FALSE: FAILED
--------------------------------------------------------------------*/
BOOL RK_WriteProvisioningData(USHORT nID,PBYTE pDataBuffer,USHORT
nBufferSize,DWORD dwLayer=0);
```

7. Linux Application Demo Code

- **Access using the "vendor_storage" node provided by the kernel**

```
#include <fcntl.h>
#include <sys/ioctl.h>

typedef unsigned short uint16;
typedef unsigned long uint32;
typedef unsigned char uint8;

#define VENDOR_REQ_TAG      0x56524551
#define VENDOR_READ_IO      _IOW('v', 0x01, unsigned int)
#define VENDOR_WRITE_IO     _IOW('v', 0x02, unsigned int)

#define VENDOR_SN_ID        1
#define VENDOR_WIFI_MAC_ID  2
#define VENDOR_LAN_MAC_ID   3
#define VENDOR_BLUETOOTH_ID 4

struct rk_vendor_req {
    u32 tag;
    u16 id;
    u16 len;
```

```
    u8 data[1];
};

int vendor_storage_read_test(void)
{
    uint32 i;
    int ret ;
    uint8 p_buf[2048]; /* malloc req buffer or used extern buffer */
    struct rk_vendor_req *req;

    req = (struct rk_vendor_req *)p_buf;
    int sys_fd = open("/dev/vendor_storage",O_RDWR,0);
    if(sys_fd < 0){
        ERROR("vendor_storage open fail\n");
        return -1;
    }

    req->tag = VENDOR_REQ_TAG;
    req->id = VENDOR_SN_ID;
    req->len = 512; /* max read length to read*/

    ret = ioctl(sys_fd, VENDOR_READ_IO, req);
    /* return req->len is the real data length stored in the NV-storage */
    if(ret){
        ERROR("vendor read error\n");
        return -1;
    }

    return 0;
}

int vendor_storage_write_test(void)
{
    uint32 i;
    int ret ;
    uint8 p_buf[2048]; /* malloc req buffer or used extern buffer */
    struct rk_vendor_req *req;

    req = (struct rk_vendor_req *)p_buf;
    int sys_fd = open("/dev/vendor_storage",O_RDWR,0);
    if(sys_fd < 0){
        ERROR("vendor_storage open fail\n");
        return -1;
    }

    req->tag = VENDOR_REQ_TAG;
    req->id = VENDOR_SN_ID;
    req->len = 32; /* data len */
    for (i = 0; i < 32; i++)
        req->data[i] = i;
    ret = ioctl(sys_fd, VENDOR_WRITE_IO, req);
    if(ret){
        ERROR("vendor write error\n");
        return -1;
    }

    return 0;
}
```

- **The user mode applications access without kernel drivers**

```
Refer to the following code for implementation:
hardware/rockchip/libvendor_storage/vendor_storage_test.c
```